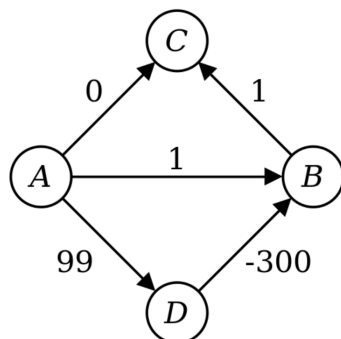


Final Review - 5/6/2026

Name all the ways you could implement a hash table that we have covered in this class (including import options) describe briefly how implementation would change for collision resolution methods and the consequences of changing these methods.

- **Separate Chaining:** A bound by linked list efficiency in the worst case scenario, no secondary collisions or clustering
- **Linear probing:** Limited by the size of the hash table, clustering effects due to collisions
- **Quadratic Probing:** Limited by the size of hash table, and primary clustering is avoided, but secondary clustering can still occur
- **Double Hashing:** Reduce clustering by using an entirely new hashing system
- We can implement a HashTable using chaining with linked lists, there are collisions appended to a chain and its time complexity is $O(n)$ in the worst case. Chaining with a BST improves the look up to $O(\log n)$ but adds balancing. We can use Java's HashMap and HashSet as import options since they use chaining

Observe the following graph.



Can we perform Dijkstra's Algorithm on this graph? Why or why not?

- We cannot do Dijkstra's Algorithm because it has a negative weight. With negative weight we can end up with a not optimal distance. Also Dijkstra's algorithm here might not reach every node. We have one way pointers, and in case we go to node C, we would be stuck there

Write a program that solves the activity selection problem using a greedy algorithm.

You are given a list of activities, where each activity has a name, a start time, and an end time. Your goal is to select the maximum number of activities that can be completed without overlapping.

Two activities are compatible if one activity ends before or exactly when the next activity begins.

```

import java.util.Arrays;

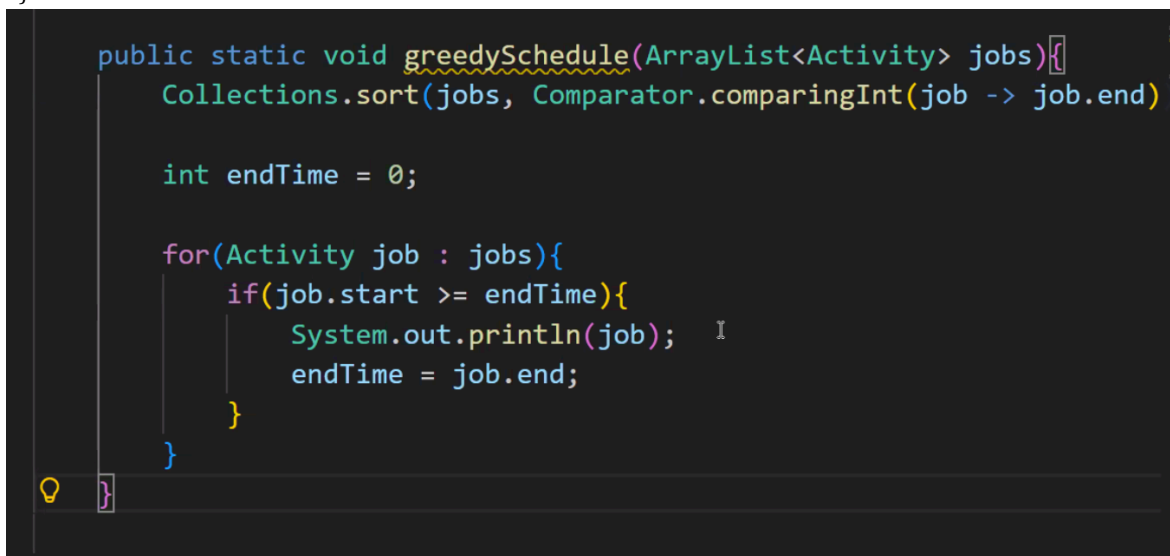
public class Activity {
    public static void main(String[] args) {
        String[] names = {"A", "B", "C"};
        int[] start = {1, 3, 0};
        int[] end = {4, 5, 6};

        Integer[] indices = {0, 1, 2, 3, 4};
        Arrays.sort(indices, (a, b) -> end[a] - end[b]);

        System.out.println("Selected activities:");
        int lastEnd = -1;

        for (int i : indices) {
            if (start[i] >= lastEnd) {
                System.out.println(names[i] + " (start: " + start[i] + ", end: " + end[i] + ")");
                lastEnd = end[i];
            }
        }
    }
}

```



```

public static void greedySchedule(ArrayList<Activity> jobs){
    Collections.sort(jobs, Comparator.comparingInt(job -> job.end)

    int endTime = 0;

    for(Activity job : jobs){
        if(job.start >= endTime){
            System.out.println(job);
            endTime = job.end;
        }
    }
}

```

Write a program that finds the largest sum you can make using numbers from a list, but you are not allowed to use two numbers that are next to each other. For example, given this list: [3, 2, 5, 10, 7] the largest sum would be $3 + 5 + 7 = 15$

```

int[] nums = new int[]{3, 2, 5, 10, 7};

int include = 0;
int exclude = 0;

for (int i = 0; i < nums.length; i++) {
    int newInclude = exclude + nums[i];
    int newExclude = Math.max(include, exclude);
    include = newInclude;
    exclude = newExclude;
}

int largest = Math.max(include, exclude);

```

```

60
61     public static int maxSum(int[] numbers){
62         if(numbers.length == 0){
63             return 0;
64         }
65         if(numbers.length == 1){
66             return numbers[0];
67         }
68
69         int[] tab = new int[numbers.length];
70         tab[0] = numbers[0];
71         tab[1] = Math.max(numbers[0], numbers[1]);
72         for(int i = 2; i < numbers.length; i++){
73             int tempInclude = numbers[i] + tab[i-2];
74             int skipInclude = tab[i-1];
75
76             tab[i] = Math.max(tempInclude, skipInclude);
77         }
78
79         return tab[numbers.length-1];
80     }

```

Given the following code snippet, determine its Big O time complexity and justify your answer:

```
def process(data):  
    for i in range(len(data)):  
        for j in range(i, len(data)):  
            print(data[i], data[j])  
    for k in range(len(data)):  
        print(data[k])
```

- I believe the Big O time complexity is $O(n^2)$. The code has nested loops, so the inner loop runs a number of times depending on i . So it would result in a quadratic complexity

I see that there is also another loop which is separated from the nested loop, that one since it is by itself it runs in linear time which makes the time complexity be $O(n)$

Write a recursive function in Java that takes a string and a character as input and returns the number of times the character appears in the string. Then, explain the base case and the recursive case of your function in plain text underneath your code submission.

Example Input/Output:

`count_occurrences("banana", "a")` # Output: 3

`count_occurrences("recursion", "r")` # Output: 2

`count_occurrences("hello", "z")` # Output: 0

`count_occurrences("", "a")` # Output: 0

Requirements:

- Your function should be fully recursive (no loops allowed).
- It should have a clear base case and recursive case.

```

public static int characterInString(String word, char character) {
    if (word.length() == 0) {
        return 0;
    }
    if (word.charAt(0) == character) {
        return 1 + characterInString(word.substring(1), character);
    } else {
        return 0 + characterInString(word.substring(1), character);
    }
}

```

- The base case for this is checking if the length of the word is equal to 0; if it is 0, which would be empty, then the program would return 0, and it stops. The recursive cases are checking the first character and then calling the function again on the remainder of the string, which is the (substring(1)). So it breaks the big task into some smaller versions of the task, checking until the string is empty to return the count.

Balanced AVL Trees are allowed to skew in one direction or another.

- True

Which of the following are true about AVL trees and Binary Search Trees as we have implemented in class?

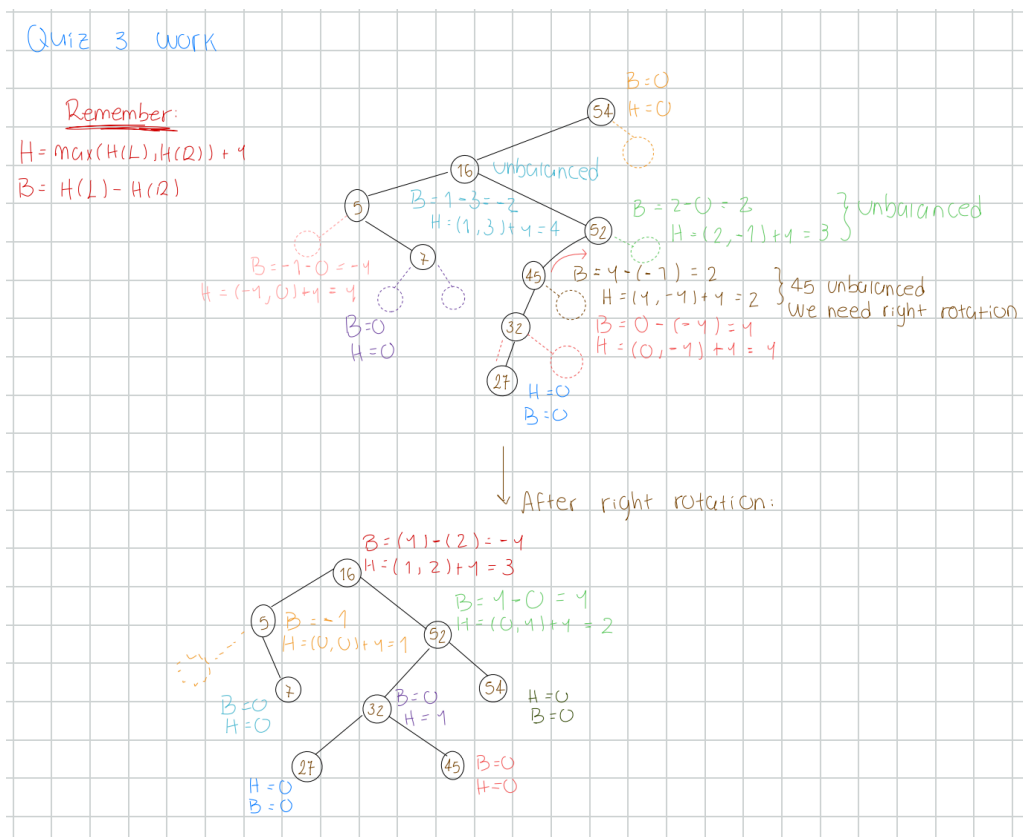
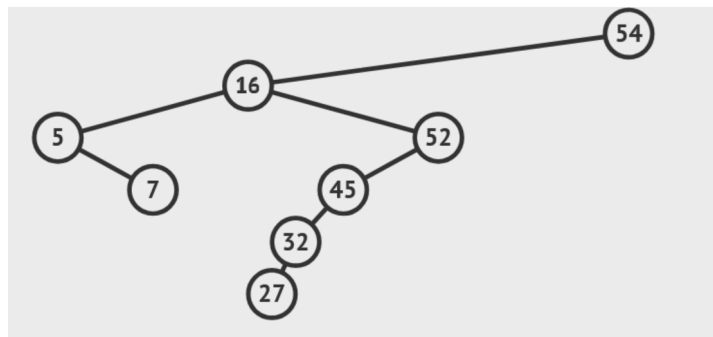
AVL trees allow duplicate values and BSTs don't

AVL trees store height information at the leaves only

Every AVL tree is also a BST

AVL Trees guarantee $O(\log n)$ efficiency on search

Using pen, pencil, or the paint program, caculate the points of imbalance in the tree below as you go through and perform rotations following AVL tree rules. Show all intermediate trees with the next point of imbalance pointed out with its balance value. Upload a picture of your final work.



In java, write a function that checks for duplicates within an array that has linear time-efficiency.

```

import java.util.Array;
import java.util.HashSet;

public static boolean duplicates (int [] array) {
    HashSet<Integer> checked = new HashSet<>();
  
```

```

for (int i = 0; i < array.length; i++) {
    if (checked.contains(array[i])) {
        return true;
    }
    checked.add(array[i]);
}
return false;
}

```

Identifying Greedy Algorithms and Examples

1. **Identify the "greedy choice" being made at each step when giving change. Why does this choice seem optimal in the moment?**
2. **Is there a situation where this approach could fail (e.g., if you didn't have access to certain coins)?**
3. **Can you think of other systems (like scheduling tasks or packing items) where a similar greedy strategy might be used? How would you identify the greedy choice in that case?**

1. The greedy choice is always taking the largest coin that fits. Let's say we owe 67 cents, then we start with a quarter, then another, then a dime, a nickel, and then finally two pennies. It feels optimal because each coin knocks out as much of the balance that we need to reach as possible.

2. But we need to consider that this can fail with different coin sets. Let's say we only have 1 cent, 3 cents, and 4 cent coins and need to make 6 cents. Then the greedy choice picks 4 + 1 + 1, which is three coins only, but 3 + 3 does it in only two and requires fewer coins. The U.S coins have a good structure for greedy to always work, but it is not guaranteed that it will in general.

3. We can see greedy approaches in scheduling, for example. If you have many tasks with different deadlines, picking one that finishes earliest frees up some time for other tasks that we need to complete, similar to how the largest coin leaves the least remaining change. But when tasks have different priorities or weights, then the greedy choice can miss the better solution, and we would need something like Dynamic Programming (which we learned today) instead. Greedy algorithms are fast and intuitive, but it doesn't guarantee always the most optimal answer.

Differences between Adjacency List & Adjacency Matrix

An Adjacency list stores each vertex and list of all the vertices connected to it. For example:

A -> B, C

B -> C

C -> A

Feature	Adjacency List	Description	Adjacency Matrix	Description
Space	$O(V + E)$	For a <u>Sparse Graph & Memory Efficiency</u> , use an Adjacency List	$O(V^2)$	For a <u>Dense Graph & Fast edge existence checks</u> , use an Adjacency Matrix
Edge	$O(\text{degree}(v))$		$O(1)$	
Add edge	$O(1)$		$O(1)$	
Remove edge	$O(V)$		$O(1)$	
Neighbor Iteration	$O(\text{degree}(v))$		$O(V)$	
Best For:	Sparse Graphs		Dense Graphs	

If a graph has 1,000 vertices and only 2,000 edges, an adjacency list is better because $V^2 = 1,000,000$ matrix entries vs. $V + E = 3,000$ list storage

Kruskal's Algorithm

Kruskal's algorithm finds a minimum spanning tree (MST)

- The MST connects all vertices, has no cycles, uses the minimum total edge weight
- Must be undirected because you assume you can travel both directions A - B (not A -> B [directed])
- Designed for undirected weighted graphs

Steps of Kruskal

1. Sort all edges by weight
2. Start with empty MST
3. Add smallest edge
4. If it creates a cycle → skip it
5. Repeat until:

$V - 1$ edges

Feature	Kruskal's	Description
Sorting Edges	$O(E \log E)$	Sorting dominates for the Kruskal's Algorithm
Union-Find Operations	$O(E \alpha(V))$	
Overall	$O(E \log E)$	
Space Complexity	$O(V + E)$	

Dynamic Programming

Let's say we are given coin values as follows:

Coins = [1,2,5]

Amount = 11

And we need to find the minimum number of coins which the answer would be $11 = 5 + 5 + 1$, with only 3 coins.

Import java.util.Arrays;

```
public class coinChange {
    public static int coinChange(int [ ] coins, int amount) {
```

```
        public static int coinChange(int[] coins, int amount) {

            int[] dp = new int[amount + 1];

            // Fill with a large value
            Arrays.fill(dp, amount + 1);

            // Base case
            dp[0] = 0;

            for(int i = 1; i <= amount; i++){

                for(int j = 0; j < coins.length; j++){

                    int coin = coins[j];

                    if(i >= coin){

                        dp[i] = Math.min(
                            dp[i],
                            dp[i - coin] + 1
                        );
                    }
                }
            }

            if(dp[amount] > amount){
                return -1;
            }
            else{
                return dp[amount];
            }
        }

        public static void main(String[] args){

            int[] coins = {1,2,5};

            int result = coinChange(coins,11);

            System.out.println(result);
        }
    }
}
```

Time Complexity

Outer Loop: amount

Inner Loop: num of coins

Total: $O(\text{amount} \times \text{numOfCoins})$

Space Complexity: $O(\text{amount})$

Greedy Algorithm

Greedy algorithm makes the best immediate choice at every step.

For example we have Kruskal, Dijkstra, Activity selection...

Lets say we have the coins: 25, 10, 5, and 1. Then we need to make 41, the greedy choses

25

10

5

1

Which is only 4 coins.

Feature	General Time Complexity of Greedy	Algorithms	Time Complexity
Sort Data	$O(n \log n)$	Kruskal	$O(E \log E)$
Process elements	$O(n)$	Prim (Heap)	$O(E \log V)$
Total	$O(n \log n)$	Dijkstra (Heap)	$O(E \log V)$
		Activity Selection	$O(n \log n)$

Differences between DP and Greedy Algorithm

Dynamic Programming (DP)	Greedy
Solves all subproblems	Makes immediate choices
Guarantees optimal answers	Sometimes optimal
Stores previous results	Does not remember previous results
Usually slower	Usually Faster

Greedy can miss the optimal solution and DP checks all relevant subproblems and guarantees correctness so it is better to use DP instead.

Differences Between DFS, BFS and Dijkstra's

Feature	DFS	Description	BFS	Description	Dijkstra's	Description
Data Structure	Stack	If we need traversal/cycle checking, then it is better to use DFS. Dive Deep.	Queue	If we need the shortest path in an unweighted graph then it is better to use BFS. Broad First.	Priority Queue	If we need the shortest path in a weighted graph then it is better to use Dijkstra's
Exploration	Deep First		Level by level		Smallest Distance First	
Finds Shortest?	No		Yes (unweighted only)		Yes (weighted)	
Weighted Graph?	Yes		Yes		Non-negative weights	
Time (Adj List)	$O(V + E)$		$O(V + E)$		$O(E \log V)$	
Space	$O(V)$		$O(V)$		$O(V)$	

DFS: Traversing graphs, detecting cycles, topological sorting, connected components, path existence. Implemented with Stack or Recursion which uses the stack.

- Adjacency List: $O(V + E)$
- Adjacency Matrix: $O(V^2)$
- Space Complexity: $O(V)$

BFS: Explores level by level. Shortest path in an unweighted graph, levels/distance, web crawlers, social network connections. Implemented with Queue.

- Adjacency List: $O(V + E)$
- Adjacency Matrix: $O(V^2)$
- Space Complexity: $O(V)$

Dijkstra's: Find shortest path from one source vertex to all others. Only works in weighted graphs and non-negative edges. Implemented with Priority Queue

- Adjacency List + Min Heap/PQ: $O((V + E) \log V)$
 - Simplified to $O(E \log V)$
- Adjacency Matrix: $O(V^2)$
- Space Complexity: $O(V)$

Data Structures	Stores	Keeps Order?	Allow Duplicates?	Best For	Time Complexity
ArrayList	Order Elements	Yes	Yes	Accessing elements by index	O(1) Constant
HashSet	Unique elements	No	No	Avoiding duplicates, fast checks	O(1) Constant
HashMap	Key-Value pairs	No	Yes (Keys are unique)	Fast Lookup by key	O(1) Constant

Data Structure	Time Complexity								Space Complexity
	Average				Worst				Worst
	Access	Search	Insertion	Deletion	Access	Search	Insertion	Deletion	
<u>Array</u>	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
<u>Stack</u>	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
<u>Queue</u>	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
<u>Singly-Linked List</u>	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
<u>Doubly-Linked List</u>	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
<u>Skip List</u>	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n \log(n))$
<u>Hash Table</u>	N/A	$O(1)$	$O(1)$	$O(1)$	N/A	$O(n)$	$O(n)$	$O(n)$	$O(n)$
<u>Binary Search Tree</u>	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
<u>Cartesian Tree</u>	N/A	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	N/A	$O(n)$	$O(n)$	$O(n)$	$O(n)$
<u>B-Tree</u>	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(n)$
<u>Red-Black Tree</u>	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(n)$
<u>Splay Tree</u>	N/A	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	N/A	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(n)$
<u>AVL Tree</u>	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(n)$
<u>KD Tree</u>	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$

Array Sorting Algorithms

Algorithm	Time Complexity			Space Complexity
	Best	Average	Worst	Worst
<u>Quicksort</u>	$O(n \log(n))$	$O(n \log(n))$	$O(n^2)$	$O(\log(n))$
<u>Mergesort</u>	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$	$O(n)$
<u>Timsort</u>	$O(n)$	$O(n \log(n))$	$O(n \log(n))$	$O(n)$
<u>Heapsort</u>	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$	$O(1)$
<u>Bubble Sort</u>	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
<u>Insertion Sort</u>	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
<u>Selection Sort</u>	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
<u>Tree Sort</u>	$O(n \log(n))$	$O(n \log(n))$	$O(n^2)$	$O(n)$
<u>Shell Sort</u>	$O(n \log(n))$	$O(n(\log(n))^2)$	$O(n(\log(n))^2)$	$O(1)$
<u>Bucket Sort</u>	$O(n+k)$	$O(n+k)$	$O(n^2)$	$O(n)$
<u>Radix Sort</u>	$O(nk)$	$O(nk)$	$O(nk)$	$O(n+k)$
<u>Counting Sort</u>	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$
<u>Cubesort</u>	$O(n)$	$O(n \log(n))$	$O(n \log(n))$	$O(n)$

Data Structure	Time Complexity					
	Storage	Add Vertex	Add Edge	Remove Vertex	Remove Edge	Query
Adjacency list	$O(V+E)$	$O(1)$	$O(1)$	$O(V+E)$	$O(E)$	$O(V)$
Incidence list	$O(V+E)$	$O(1)$	$O(1)$	$O(E)$	$O(E)$	$O(E)$
Adjacency matrix	$O(V^2)$	$O(V^2)$	$O(1)$	$O(V^2)$	$O(1)$	$O(1)$
Incidence matrix	$O(V \cdot E)$	$O(V \cdot E)$	$O(V \cdot E)$	$O(V \cdot E)$	$O(V \cdot E)$	$O(E)$

	BFS	DFS
Definition	Breadth-First Search	Depth-First Search
Implementation	Queue	Stack and Recursion
Used for	To find shortest path	To exhaustively search all possibilities
Suitability	If a solution is not far from the root node. If you need to search part of the tree for a solution. If the depth of the tree vary.	If a solution is far from the root node. If you have to search the entire tree to find a solution.
Time Complexity	$O(V+E)$	$O(V+E)$

Data Structures

Data Structure	Time Complexity								Space Complexity
	Average				Worst				Worst
	Indexing	Search	Insertion	Deletion	Indexing	Search	Insertion	Deletion	
Basic Array	$O(1)$	$O(n)$	-	-	$O(1)$	$O(n)$	-	-	$O(n)$
Dynamic Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Singly-Linked List	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Doubly-Linked List	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Skip List	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n \log(n))$
Hash Table	-	$O(1)$	$O(1)$	$O(1)$	-	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Binary Search Tree	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Cartesian Tree	-	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	-	$O(n)$	$O(n)$	$O(n)$	$O(n)$
B-Tree	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(n)$
Red-Black Tree	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(n)$
Splay Tree	-	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	-	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(n)$
AVL Tree	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(n)$

BFS

- Use when you need to search the shortest path.
- The node you are looking for is not far from the root node.
- It is necessary to traverse the parts of the graph.
- When there is a difference in depth on the graph.
- Requires more **memory** than DFS because you have to store pointers of neighboring nodes.

DFS

- Use to search the entire graph.
- The node you are looking for is far from the root node.
- It is recommended to use it when there is not much difference in depth on the graph.